

Proposed Solution Report: Cloud-Based Movie Streaming Platform Using FastAPI and Next.js

Mustakim Ahmed Hasan (2421349042), MD. Nafiz Al Mamun (2232050642),
Tanim Ahmed (2312597042), Ahamed Osman Asif (2031080642)
CSE299.4; Group: 01
Department of Computer Science and Engineering
North South University

I. PROBLEM CONTEXT

This project develops a Netflix-style cloud movie streaming platform featuring authentication, HLS playback, watch history, trending logic, and cloud deployment. Initial implementation suffered from unstable deployments, backend runtime mismatches, incorrect routing behavior, data inconsistencies, and frontend build/type failures.

II. KEY CHALLENGES

Major issues included Git push rejections from large tracked artifacts, inconsistent deployment between local and cloud (Vercel/Render), FastAPI endpoint mismatches, missing static directories in serverless runtime, PostgreSQL SSL/connection errors, route conflicts (e.g., `/movies/trending` vs. `/movies/{id}`), password hashing incompatibility, incorrect trending computation, incomplete history payloads, TypeScript build failures, CORS instability, and non-reactive authentication state handling.

III. PROPOSED ARCHITECTURE IMPROVEMENTS

Infrastructure and Version Control: Strict artifact exclusion, removal of large binaries from Git history, use of Supabase Storage for media assets, and clean commit separation for infra and application changes.

Backend Stabilization: Normalized FastAPI routing and endpoints, dependency pinning, cloud-safe startup without filesystem assumptions, corrected trending computation using `watch_history`, enriched history responses, and validated PostgreSQL connection configuration.

Frontend Reliability: Enforced strict API response typing for TypeScript, centralized authentication state updates, route-level loading states, improved player controls (fullscreen, keyboard shortcuts, inactivity overlay), and stable CORS configuration.

Streaming Strategy: Adoption of HLS format (master.m3u8, variant playlists, TS segments), storage in structured Supabase buckets, and persistence of metadata plus public HLS URLs in PostgreSQL.

IV. IMPLEMENTATION PLAN

- 1) Stabilize backend endpoints and database connectivity.
- 2) Finalize authentication/session behavior and deployment CORS policy.
- 3) Integrate reliable watch-history tracking from player events.
- 4) Complete UI/UX improvements across desktop and mobile.
- 5) Execute full-route smoke testing prior to submission.

V. EXPECTED OUTCOME

The refined system delivers stable deployment, accurate trending logic, reliable authentication handling, cloud-based HLS streaming, and a production-ready architecture suitable for academic demonstration and future scalability.

VI. CONCLUSION

The proposed restructuring transforms the project into a maintainable, deployment-ready streaming platform by consolidating infrastructure, backend correctness, and frontend stability into a unified cloud architecture.